

SUSE Observability

Guia simples de estudo

Helm e Fleet • versão 6 • setembro de 2026

Instalar, entender e operar. Três charts, perfis curtos de values e detalhes técnicos separados. Material em português; dashboards e README em inglês.

Use os arquivos Markdown do pacote para copiar comandos sem quebras visuais. Preencha caminhos, contextos, endpoints, nomes de integração e credenciais próprias antes da implantação.

Comece por este roteiro. Consulte docs/details quando precisar aprofundar. Os resultados do laboratório e os testes necessários no cliente estão separados em docs/VALIDATION.md. Este pacote customizado não substitui a documentação oficial do produto.

Guia de estudo: entenda o projeto

Leia este guia para compreender a implementação. Para instalar, use o [procedimento do cliente](#). Os detalhes de cada configuração ficam na [pasta técnica](#).

1. O caminho da informação

A origem produz uma métrica; um coletor a lê; o SUSE Observability recebe e armazena; o dashboard consulta. Um monitor avalia uma condição. Uma notificação decide quem recebe o aviso.

Exemplo: o Traefik conta respostas HTTP 500. O coletor envia o contador ao SUSE. O dashboard mostra quando ocorreram erros e qual aplicação foi afetada. O monitor avalia a taxa de erros. Uma regra de e-mail avisa a equipe quando as condições configuradas forem atingidas.

Instalar apenas o dashboard cria a visualização, mas não cria métricas que a origem ainda não oferece.

2. Por que existem três charts

Chart	Onde fica	O que instala
sre-platform-telemetry	Uma vez por cluster	Coleta do SO, etcd, Traefik e inventário de configurações de Pods
sre-kubevirt-telemetry	Uma vez no Harvester	Collector OpenTelemetry para métricas de VMs/KubeVirt
suse-observability-content	Uma vez no central	Dois Jobs que publicam todos os dashboards e monitores selecionados

O conteúdo é centralizado para evitar repetir URL e credenciais por dashboard. Os coletores ficam separados porque executam em lugares diferentes. O chart central não precisa de kubeconfig de todos os clusters; ele consulta os dados que já chegaram ao servidor.

Os Jobs de publicação terminam. Os coletores continuam executando. Nenhum desses charts instala o servidor SUSE nem cria PVCs próprios.

3. O que cada arquivo faz

```
charts/nome-do-chart/  
  Chart.yaml      nome e versão do pacote  
  values.yaml     configuração padrão completa  
  values.schema.json validação dos valores  
  templates/      modelos dos recursos Kubernetes  
  files/          código e arquivos auxiliares  
  fleet.yaml      como Fleet direciona a instalação  
  config/         exemplos usados pelo Fleet/laboratório  
  
deploy/  
  cluster/values.yaml      perfil curto de plataforma  
  harvester/values.yaml    perfil curto de plataforma Harvester  
  virtual-machines/values.yaml perfil de VMs  
  central/values.yaml      perfil de conteúdo
```

Helm combina os defaults com o arquivo passado em `-f`. Portanto, seu perfil precisa conter apenas o que muda. Requests e limits continuam vindo dos defaults. O `values.schema.json` rejeita várias configurações inválidas antes do deploy.

`helm template` mostra o manifesto resultante sem instalar. `helm upgrade --install` cria ou atualiza a release. `helm uninstall` remove os recursos Kubernetes da release; conteúdo gravado numa API externa precisa de uma etapa própria de exclusão.

4. Por que usar token e Secret

Termo	Significado
Role	Conjunto de permissões no SUSE
Service Token	Identidade utilizada pela automação
Kubernetes Secret	Local onde o Pod encontra a credencial
Token de ingestão	Permite enviar dados; não é o publicador
Token de publicação	Permite criar/atualizar o conteúdo SUSE

Para um cliente novo, a role padrão `stackstate-k8s-troubleshooter` pode evitar a criação de roles próprias, após verificar as permissões da versão. Ela tem um escopo operacional maior. Roles dedicadas são a alternativa de menor privilégio. Os dois Jobs podem usar o mesmo Secret, embora a instalação anterior tenha usado dois.

O `sts` é a CLI do SUSE Observability. Um administrador o utiliza para preparar o Service Token uma vez. Depois, o chart executa sua própria CLI dentro dos Jobs. Isso não concede acesso ao assistente: é uma credencial persistente da automação do cliente, que deve ter responsável, validade e rotação.

Numa atualização, preserve a identidade proprietária dos dashboards. Mesmo conjunto de permissões não transforma outro token no mesmo proprietário. Veja [preparo do publicador](#).

5. Como interpretar as métricas

Painel/medida	Leitura correta
Requests/s em cinco minutos	Tráfego dividido por uma janela móvel de 300 segundos
Total no período selecionado	Estimativa por contadores no intervalo da barra superior; não é contagem de access logs
Tempo médio de resposta	Soma dos tempos dividida pelo número de chamadas
p95	Estimativa do tempo em que 95 de cada 100 chamadas terminam
CPU do Pod	Soma do consumo dos containers regulares; 1000 millicores equivalem a um núcleo
Memória do Pod	Working set dos containers; requests/limits vêm da configuração Kubernetes
Mais de 80% do limit	Sinal para investigar, não prova automática de falta de recurso
Memória da VM	Identificar se vem do SO convidado ou do hipervisor; são medidas diferentes
Disco do host	Filesystem montado relevante; uso do PVC e ocupação física Longhorn podem diferir
Fila de envio igual a zero	Nenhum dado aguardando envio naquele instante; normalmente saudável

Um teste de 200 chamadas em dois segundos alcança 100 req/s durante o teste. Distribuído numa média de cinco minutos, esse tráfego representa cerca de 0,67 req/s. Compare a mesma aplicação, período e ponto de medição. O navegador inclui conexão/rede; Traefik e instrumentação medem trechos diferentes.

Um histograma estima percentis por faixas. Por isso, p95 do dashboard não precisa coincidir com o percentil calculado a partir de cada duração registrada pelo navegador. Aguarde coleta/ingestão e atualize a tela antes de comparar totais.

Métricas ajudam a localizar aplicação, status e intervalo do erro. Para explicar a causa de uma requisição específica, correlacione logs/traces com horário e identificador. Evite adicionar URLs com IDs livres ou payloads às labels.

6. Como foi construído um dashboard

1. Definir a pergunta: por exemplo, quais Pods estão próximos do limite de memória?
2. Encontrar a métrica real e seus rótulos no Metrics Explorer.
3. Validar unidade e origem contra Kubernetes/SO. Não inferir configuração pelo consumo.
4. Escrever a consulta e testar um cluster, todos os clusters e filtros sem correspondência.
5. Escolher a apresentação: número para quantidade, lista para nomes, curva para evolução, gauge para uma proporção.
6. Escrever título curto e descrição com unidade, janela, interpretação e próxima ação.
7. Exportar a definição compatível com a versão SUSE, incluir no chart e validar o conteúdo salvo após publicar.

As definições deste projeto estão em `charts/suse-observability-content/dashboards/`. O schema foi verificado na versão instalada; não basta inventar um tipo de widget ou campo. A interface 2.10.2 tem limitações de tabelas/legendas documentadas nos testes.

Ao editar a consulta de um dashboard, atualize sua descrição, unidade, filtros e relação com o monitor. Se adicionar uma nova família de métricas, reveja também o coletor, a allowlist e o impacto de cardinalidade.

7. Como criar ou alterar um chart

Em uma pasta de estudo separada:

```
helm create meu-exemplo
helm lint ./meu-exemplo
helm template estudo ./meu-exemplo > /tmp/estudo.yaml
```

Abra `Chart.yaml`, `values.yaml` e `templates/`. Remova recursos que seu componente não precisa; escolha imagem, requests/limits, probes, ServiceAccount, permissões e exposição de rede. O scaffold é ponto de partida, não aprovação de produção.

Para evoluir este projeto: altere o `template/default`, ajuste o schema se necessário e renderize com um perfil curto. Somente depois valide no cluster canário. Os testes de desenvolvimento ficam separados da instalação do cliente:

```
python -m pip install -r requirements-dev.txt
python -m unittest discover -s tests -v
bash tests/helm-checks.sh
```

Esses comandos verificam código e renderização. Não substituem pull real de imagens, autenticação, métricas corretas, filtros ou recebimento de e-mails.

8. Monitores e boas práticas

Um monitor útil identifica recurso, condição, duração e ação. Use sinais de impacto e saturação, não um alerta para cada gráfico. Combine erro/latência HTTP, disponibilidade, pressão de recursos, armazenamento, etcd e integridade da coleta. Preserve monitores nativos que já cobrem o problema.

Um limiar inicial, como 80% do limit ou 20 ms de disco, é um ponto de investigação. Ajuste ao SLO e baseline do cliente. Defina janela suficiente para evitar ruído; teste abertura, recuperação, destinatários e duplicidade entre alertas de aplicação e ingress.

Os 33 monitores estão em `charts/suse-observability-content/monitors/`. O catálogo seleciona quais publicar; [ALERT-COVERAGE.md](#) liga painéis a alertas. As notificações são configuradas na interface, não pelo chart.

9. Desempenho, segurança e limites

As coletas usam intervalos de 30s, allowlists, filas limitadas e requests/limits. Mais séries, labels, retenção e consultas simultâneas aumentam custos. Um gráfico extra da mesma série não duplica o armazenamento dessa série.

No laboratório, os coletores adicionais somaram cerca de 69,7 mCPU médios e 730,4 MiB ao somar os maiores usos de memória de cada cluster na amostragem curta. Esses números não são requests recomendados nem pico garantido. Não existia baseline controlado antes da primeira instalação. O [relatório simples](#) explica como planejar o cliente e onde consultar os números completos.

Host networking e mounts read-only de host continuam exigindo avaliação de segurança. Use TLS verificado, credenciais fora do Git, fontes confiáveis e rollout canário. Não prometa impacto zero: confirme comportamento com a carga e as políticas do destino.

Para aprofundar

Abra [docs/details](#) apenas quando precisar de internals, fórmulas completas, SAN/CA, roles restritas, migração, dimensionamento ou diagnóstico. A [documentação oficial SUSE](#) continua sendo a referência de suporte e operação do produto.